Gradient-Free Optimization

Interval Search, Nelder–Mead, and Simulated Annealing

1 Searching using function values

Suppose we can calculate a loss or negative log-likelihood at proposed parameter values. A *gradient-free* method searches for small values by comparing these evaluations, without requiring derivatives.

Think of exploring a landscape while only knowing your height at the places you visit. With one parameter, we narrow an interval; with a few parameters, we move a collection of points. With several valleys, we may accept an uphill move to explore another valley.

These methods are useful when derivatives are inconvenient or unavailable. The main question is: *where should we evaluate the objective next?*

2 Trisection: discard part of an interval

Suppose we want to find the smallest value of $f(x)$ on an interval $[a, b]$. We can evaluate f at any chosen point, but we do not use derivatives. The idea is to keep an interval containing the minimum and make that interval progressively smaller.

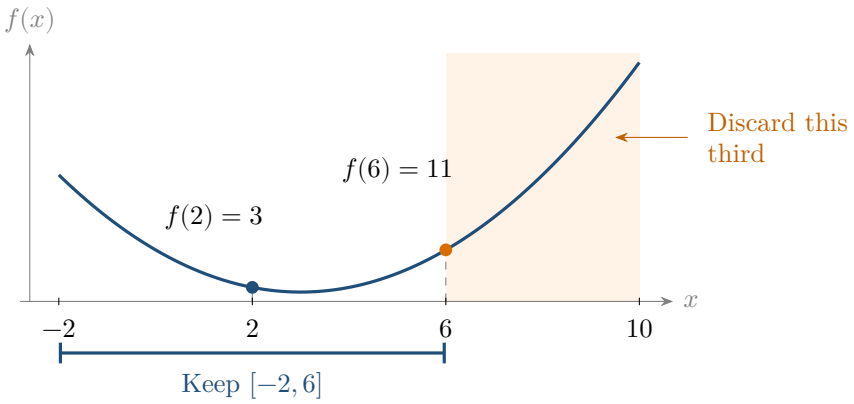
For now, assume the function has **one clear valley**: it strictly decreases toward its minimum and strictly increases afterward. This is the unimodal shape we use below. Write $x^\star$ for the unknown location of the minimum.

Where do we look? Evaluate the function at two points that divide the current interval into thirds:

$$x_1 = a + \frac{b-a}{3}, \qquad x_2 = a + \frac{2(b-a)}{3}.$$

The comparison between $f(x_1)$ and $f(x_2)$ tells us which part of the interval can be discarded.

- If $f(x_1) < f(x_2)$, keep $[a, x_2]$. To see why, imagine that the minimum were to the right of x_2. Both test points would then be on the decreasing side of the valley, requiring $f(x_1) > f(x_2)$. The observed comparison rules this out. The minimum can still be between the two points or farther to the left.
- If $f(x_1) > f(x_2)$, keep $[x_1, b]$. If the minimum were to the left of x_1, both points would be on the increasing side, contradicting the observed comparison.
- If $f(x_1) = f(x_2)$, keep $[x_1, x_2]$. Under the stated strict decrease and increase, two distinct points with equal heights must straddle the minimum.



A numerical step. For $f(x) = (x-3)^2 + 2$ on $[-2, 10]$, the test points are 2 and 6. Since $f(2) = 3 < f(6) = 11$, retain $[-2, 6]$. Divide this new interval into thirds again: the new test points are $2/3$ and $10/3$. Their function values are approximately 7.44 and 2.11, so the next retained interval is $[2/3, 6]$.

When do we stop? Each unequal comparison retains two-thirds of the current interval. Stop when its width is below a chosen tolerance. If the retained interval has width w, its midpoint is at most $w/2$ away from $x^\star$. This gives an accuracy statement about the *location* of the minimum.

3 Golden-section search: reuse an evaluation

Trisection generally needs two new function values after each interval reduction. The old test points usually do not fall at the one-third and two-thirds positions of the smaller interval. If evaluating the objective is expensive, repeatedly replacing both points can be costly.

Golden-section search chooses the positions so that **one old point is already in the correct position for the next comparison**. Its function value can be stored and reused.

Where do we look? Set $\tau = (\sqrt{5} - 1)/2 \approx 0.618$ and use

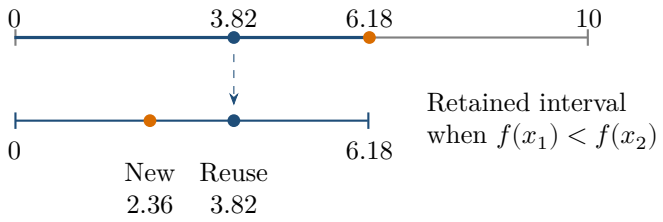
$$x_1 = a + (1 - \tau)(b - a), \qquad x_2 = a + \tau(b - a).$$

These points lie approximately 38.2% and 61.8% of the way from a to b. Apply the same comparison rule as in trisection.

What gets reused? For illustration, start with $[0, 10]$, so the test points are approximately 3.82 and 6.18. Suppose $f(3.82) < f(6.18)$. Keep $[0, 6.18]$. The two required positions within this smaller interval are approximately

$$0 + 0.382(6.18) \approx 2.36, \qquad 0 + 0.618(6.18) \approx 3.82.$$

The old left point, 3.82, becomes the new right point. Its location has stayed the same; its role within the interval has changed. We already know $f(3.82)$, so only $f(2.36)$ needs to be evaluated.



If instead $f(x_1) > f(x_2)$, keep $[x_1, b]$: the old right point becomes the new left point. For a tie, either of these two intervals can be kept under our one-valley assumption, preserving the reuse pattern.

Why these fractions? Write $L = b - a$. When we retain the left interval, its length is τL. Its new right test point is therefore $a + \tau(\tau L) = a + \tau^2 L$. To make this equal the old left test point, $a + (1 - \tau)L$, choose τ so that

$$\tau^2 = 1 - \tau, \qquad \text{giving} \qquad \tau = \frac{\sqrt{5} - 1}{2}.$$

Thus the fractions are chosen to make the point positions match after shrinking the interval. Reuse is exact with this value of τ; the decimal numbers above are rounded for illustration.

After the first two evaluations, each further reduction requires only one new evaluation and retains approximately 61.8% of the interval. As in trisection, stop when the interval is sufficiently narrow.

4 Brent: combine interval search with a useful guess

Golden-section search decides where to look using the interval geometry and comparisons of function values. Near a smooth minimum, the *numerical heights* of evaluated points can provide more information: they can suggest where the bottom of the valley lies.

Use a parabola to propose the next point. A smooth function near a minimum often resembles a parabola. Given three distinct evaluated points, fit a quadratic curve through their function values. For a fitted curve $q(x) = Ax^2 + Bx + C$ with $A > 0$, its bottom is at $u = -B/(2A)$. This gives a candidate point at which to evaluate f. The coefficients come from the three function values, so no derivatives of f are needed.

For example, for $f(x) = (x-3)^2 + 2$, suppose we have evaluated $f(0) = 11$, $f(2) = 3$, and $f(6) = 11$. The parabola through these values is $q(x) = x^2 - 6x + 11$, whose bottom is at $u = 3$. In this example the function is itself quadratic, so the proposal lands directly at the minimum. For a general smooth function, the fitted parabola gives an approximation that can improve as the search approaches the minimum.

Why keep the golden-section step? Three points can give a poor picture of the function away from them. A fitted parabola may open downward, suggest a point outside the interval, or propose a step that is unsuitable relative to recent progress. Brent's method uses a parabolic proposal only when its safeguards accept the step. Otherwise, it takes a golden-section step within the current interval (Brent, 1973).

How does the interval shrink? The method maintains the current interval and a best evaluated point x. Suppose the new trial point u is to the right of x. If $f(u) > f(x)$, the minimum must be to the left of u, so u becomes the right endpoint. If $f(u) < f(x)$, the minimum must be to the right

of x, so x becomes the left endpoint and u becomes the best point. A trial to the left uses the mirror-image rules. Equal values also permit an interval reduction under our stated assumption. Thus a trial can narrow the interval even when it does not improve the best function value.

The interval reduction protects the search; parabolic interpolation can accelerate it when the fitted shape is useful.

Using R. For a bounded problem with one scalar parameter, use `optimize()`, which combines golden-section search and parabolic interpolation:

```
f <- function(x) (x - 3)^2 + 2
br <- optimize(f, interval = c(-2, 10))
br$minimum       # location of the minimum: approximately 3
br$objective     # function value there: approximately 2
```

The equivalent `optim()` interface uses `method="Brent"` and finite bounds:

```
optim(0, f, method = "Brent", lower = -2, upper = 10)
```

A practical check. Plot the objective and mark the returned point:

```
curve(f, from = -2, to = 10, ylab = "f(x)", lwd = 2)
points(br$minimum, br$objective, pch = 19, col = "blue")
c(left = f(-2), found = br$objective, right = f(10))
```

The endpoint comparison is useful when the minimum could be at a boundary. If the plot shows several valleys, search the relevant intervals separately and compare the resulting function values; a single Brent search need not find the lowest valley.

5 Nelder–Mead: move and reshape a triangle

With one parameter, a trial point is a location on a line. With two parameters, it is a pair (x_1, x_2), so the search takes place in a plane. Think of $f(x_1, x_2)$ as the height of a landscape: our aim is to find a point with a low height. We can evaluate that height at any trial point without calculating a derivative.

Nelder–Mead keeps a **triangle of three trial points** and uses their function values to decide where to look next. The triangle can move, stretch, and become smaller as the search proceeds. In d dimensions, the corresponding set has $d + 1$ vertices and is called a *simplex*. For example, a simplex is a line segment in one dimension, a triangle in two dimensions, and a tetrahedron in three dimensions.

Set up the triangle. Choose three points that do not lie on the same line and evaluate the objective at each one. Label them B, G, and W so that

$$f(B) \le f(G) \le f(W).$$

Here B is the best vertex, G is the other good vertex, and W is the worst. These labels refer to *function values*, not to the horizontal or vertical positions of the points in the diagram. Each vertex is a complete candidate value of the two-dimensional parameter.

The first goal is to replace W with a more promising point. To choose a direction, take the center of the two better vertices:

$$C = \frac{B+G}{2}.$$

Thus, C is the midpoint of the edge joining B and G. We leave W out of this average because we want to use the better part of the triangle as the base for a move away from the poorest vertex. In higher dimensions, C is the average of all vertices except the worst. It is a geometric reference point; the method does not need to evaluate $f(C)$ to construct these moves.

Four possible moves. The following description uses the usual reflection, expansion, contraction, and shrink factors 1, 2, 1/2, and 1/2. Every iteration begins by trying a reflection. What happens next depends on the value found there.

1. **Reflect: look across the better edge.**

   Move from W toward the midpoint C, then continue along the same line for an equal distance beyond C. The reflected point lies on the opposite side of C, at the same distance from C:

   $$R = C + (C - W) = 2C - W.$$

   This is a trial in the direction from W through C: if the side containing W is relatively poor, the other side may be more promising.

   Evaluate $f(R)$. If $f(B) \le f(R) < f(G)$, replace W by R. The new point is better than both G and W, although it does not improve on B. This is still useful: we have improved the set of trial points and moved the triangle into a more promising region.

2. **Expand: follow an especially promising reflection.**

   If $f(R) < f(B)$, the reflection has found a new best value. This suggests trying a larger move in the same direction:

   $$E = C + 2(R - C).$$

   The expanded point is twice as far from C as the reflected point. Evaluate $f(E)$. If $f(E) < f(R)$, replace W by E; otherwise replace W by R.

   The expansion is an opportunity to travel faster when a direction looks good. We keep the reflected point available because moving farther can overshoot a useful region. A good direction does not mean that every longer step in that direction is better.

3. **Contract: try a shorter move.**

   If $f(R) \ge f(G)$, reflection has not done well enough to be accepted immediately. There are two different situations.

   Outside contraction: if $f(G) \le f(R) < f(W)$, the reflection improves on the worst vertex but remains worse than the two better ones. Try the midpoint between C and R:

   $$Q_{\text{out}} = C + \frac{1}{2}(R - C).$$

   This tests the reflected side with a shorter step. Accept it if $f(Q_{\text{out}}) \le f(R)$; otherwise proceed to a shrink.

   Inside contraction: if $f(R) \ge f(W)$, even the reflected point is no improvement on the worst vertex. Try the midpoint between C and the old worst vertex:

   $$Q_{\text{in}} = C + \frac{1}{2}(W - C).$$

This stays on the original side but moves closer to the better edge. Accept it if $f(Q_{\text{in}}) < f(W)$; otherwise proceed to a shrink.

The words *outside* and *inside* refer to the original triangle. Only one of these two contraction points is tried, according to the reflection result.

4. **Shrink: search more closely around the best point.**

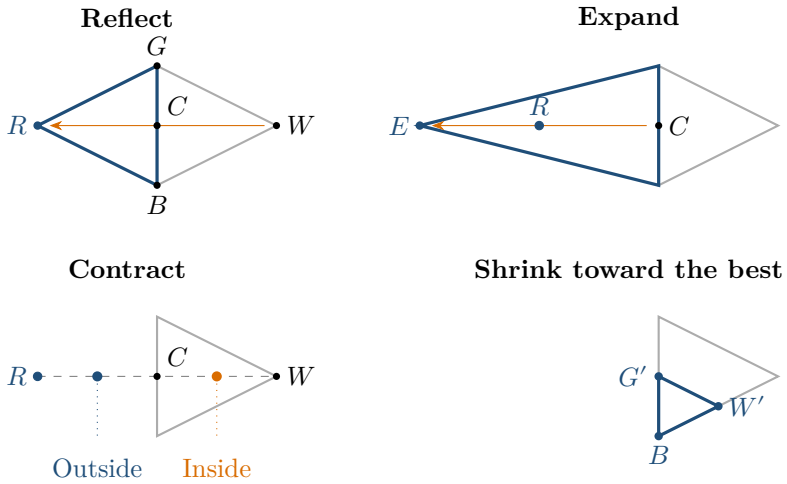
   If the chosen contraction also fails, keep B and move each of the other vertices halfway toward it:

$$G' = B + \frac{1}{2}(G - B), \qquad W' = B + \frac{1}{2}(W - B).$$

   Evaluate the objective at G' and W'. The resulting triangle explores a smaller neighborhood of the best point found so far.

   A shrink changes two vertices in two dimensions; the other moves replace only the worst vertex. The new points need not individually improve on the vertices they replace, but B remains available. Shrinking reduces the scale of exploration after the previous proposals have failed.

Repeat with new labels. After accepting a move, compare the three current function values again. Relabel the vertices, calculate the new center excluding the current worst vertex, and start another reflection. A point called B in one iteration may be called G later when a better point has been found. The four moves are conditional choices, not four operations carried out in order at every iteration.



Read the diagrams geometrically. Gray shows the old triangle. Blue shows a proposed replacement triangle or trial point; orange also marks the inside contraction. Each panel starts from the same old triangle so that the moves can be compared. These are four possible moves, rather than four successive iterations. The horizontal and vertical positions are parameter values; the objective values are not displayed as heights.

A numerical reflection with an acceptance decision. Consider $f(x_1, x_2) = (x_1 + 1)^2 + x_2^2$ and the three vertices in the reflection diagram:

$$B = (0, 0), \quad G = (0, 2), \quad W = (2, 1).$$

Their function values are $f(B) = 1$, $f(G) = 5$, and $f(W) = 10$, so the labels are correctly ordered. The center is $C = (0, 1)$ and reflection gives $R = 2C - W = (-2, 1)$.

Geometry tells us where to try; evaluating the objective tells us what to keep. Here $f(R) = (-2 + 1)^2 + 1^2 = 2$. Since $1 < 2 < 5$, accept the reflection. The old triangle had values $1, 5, 10$; the new triangle has values $1, 5, 2$. Its worst value is now 5 rather than 10, even though the best value remains 1.

For the next iteration, the old B remains best, R becomes the other good vertex, and the old G becomes worst. The new center is therefore the midpoint of $(0, 0)$ and $(-2, 1)$, namely $(-1, 1/2)$. Recomputing this center allows later moves to change direction.

How this differs from an interval search. Under a unimodality assumption, an interval method can rule out part of a line while retaining a minimizer. Nelder–Mead uses the triangle to choose new trial points; the triangle is not a guaranteed enclosure of a minimizer. Its appeal is that it can explore several parameters using function values alone.

6 Two examples in R

6.1 One parameter: estimate a Poisson mean

Suppose $Y_1, \ldots, Y_n$ are independent Poisson counts with common mean $\lambda > 0$. A candidate λ gives a probability to each observed count. Maximum likelihood chooses the value that makes the observed data most plausible.

Because the numerical routine minimizes by default, we use the *negative* log-likelihood. After dropping terms that do not depend on λ, this objective is

$$q(\lambda) = n\lambda - \left(\sum_{i=1}^{n} y_i\right) \log \lambda.$$

Dropping those terms changes the height of the curve by a constant, but it does not change where the minimum occurs.

Build the positivity constraint into the parameter. Instead of searching directly over positive λ, write $\eta = \log \lambda$, so $\lambda = \exp(\eta)$. Every real η produces a positive mean. The objective supplied to R is then

$$q(\eta) = n \exp(\eta) - \left(\sum_{i=1}^{n} y_i\right) \eta.$$

It is the same likelihood expressed in a different coordinate.

```
y <- c(0, 1, 3, 2, 4, 1, 2, 5, 2, 3)

nll <- function(eta) {
  length(y) * exp(eta) - sum(y) * eta
}

poi <- optimize(nll, interval = c(-6, 6))

eta_hat <- poi$minimum
lambda_hat <- exp(eta_hat)

c(eta_hat = eta_hat,
  numerical = lambda_hat,
  known_answer = mean(y))
```

Read the code in the order of the statistical problem. The vector `y` contains the data. The function `nll` takes one proposed value of η and returns one number measuring its fit: smaller is better. The call to `optimize()` repeatedly evaluates this function within the supplied interval.

The component `poi$minimum` is the estimated *argument* η, not the minimum objective value. The latter is stored in `poi$objective`. We exponentiate the estimated argument to return to the Poisson mean scale.

Here $n = 10$ and $\sum_i y_i = 23$. The known maximum-likelihood estimate is $\hat{\lambda} = \overline{y} = 2.3$, so the numerical answer should be close to 2.3 and $\hat{\eta}$ close to $\log(2.3) = 0.833$. This provides a direct check on both the objective and the transformation. The search interval $[-6, 6]$ is on the η scale; it allows means from $\exp(-6) \approx 0.0025$ to $\exp(6) \approx 403.4$.

See the curve being minimized. The following plot uses a narrower interval to make the valley easier to see. It does not change the interval used in the search.

```
curve(nll(x), from = -1, to = 2,
      xlab = expression(eta == log(lambda)),
      ylab = "Negative log-likelihood (up to a constant)")
abline(v = eta_hat, lty = 2, col = "steelblue")
points(eta_hat, nll(eta_hat), pch = 19,
       col = "steelblue")
```

The marked point is the bottom of the curve on the log-mean scale. The objective can be negative because we dropped a constant; its location is what matters. R's `optimize()` combines golden-section search with parabolic interpolation, connecting this example to the interval methods discussed earlier.

6.2 Two parameters: search along a curved valley

The Rosenbrock function is

$$f(x_1, x_2) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2.$$

The first term penalizes distance from the parabola $x_2 = x_1^2$. Its multiplier 100 makes movement across that curve expensive. The second term favors $x_1 = 1$. Along the parabola, the first term is zero, and the remaining term decreases as x_1 approaches 1.

Both terms are nonnegative and both vanish at $(1, 1)$, so the global minimum is $f(1, 1) = 0$. The

resulting landscape has a narrow valley that bends toward this point. Finding a low part of the valley is easier than following its bend all the way to the minimum.

```
rosenbrock <- function(x) {
  100 * (x[2] - x[1]^2)^2 + (1 - x[1])^2
}

start <- c(-1.2, 1)

nm <- optim(start, rosenbrock, method = "Nelder-Mead",
            control = list(maxit = 5000, reltol = 1e-12))

nm$par                        # estimated parameter pair
nm$value                      # objective at that pair
nm$counts                     # objective/gradient call counts
nm$convergence                # completion code

rosenbrock(start)              # 24.2
rosenbrock(c(1, 1))            # 0
```

What the optimizer receives. The function takes a vector of length two: `x[1]` is the current candidate for x_1, and `x[2]` is the candidate for x_2. The vector `start` supplies an initial parameter pair. R constructs an initial simplex from that pair. No derivatives are supplied or needed by Nelder–Mead.

Starting from a value of 24.2, the search should return a parameter pair close to $(1, 1)$ and an objective value close to zero. The triangle can reflect along the valley, expand when progress is good, and contract or shrink when a move crosses a steep side.

Read the returned quantities together. The vector `nm$par` is the best parameter pair found. The number `nm$value` measures how well that pair minimizes the objective. The `function` entry of `nm$counts` records objective evaluations; one iteration may require several evaluations to compare a reflection with an expansion or contraction.

A convergence code of `0` reports successful completion under the routine's stopping rule. A code of `1` means that the iteration limit was reached, and `10` indicates simplex degeneracy. Here the known answer lets us check the returned point directly as well.

Draw the curved valley. A contour joins locations with the same objective value. The following plot uses $\log\{1 + f(x_1, x_2)\}$ so that both small and large values can be seen on one plot. This transformation preserves the minimizer and the shapes of the contour sets; the optimization above still uses f.

```
x1 <- seq(-2, 2, length.out = 160)
x2 <- seq(-1, 3, length.out = 160)
z <- outer(x1, x2, function(a, b) {
  100 * (b - a^2)^2 + (1 - a)^2
})

contour(x1, x2, log1p(z), nlevels = 20, asp = 1,
        xlab = expression(x[1]), ylab = expression(x[2]),
        main = "Contours of log(1 + f)")
lines(x1, x1^2, lty = 2, col = "gray50")
points(start[1], start[2], pch = 19, col = "firebrick")
points(nm$par[1], nm$par[2], pch = 19, col = "steelblue")
points(1, 1, pch = 4, cex = 1.5)
legend("topleft",
       legend = c("Start", "Returned point", "Known minimum"),
       pch = c(19, 19, 4),
       col = c("firebrick", "steelblue", "black"),
       bty = "n")
```

The dashed parabola helps locate the valley. The plot marks the starting and returned points; it does not display intermediate triangles.

Practical lessons from the examples

Why scaling matters. Suppose one parameter is naturally around 1000 and the other around 0.01. A change of one unit is tiny relative to the first parameter and enormous relative to the second. The triangle's geometry can therefore favor exploration of one coordinate.

Rescale to $u_1 = x_1/1000$ and $u_2 = x_2/0.01$ so that typical values are both around one. In `optim()`, the corresponding option is `control=list(parscale=c(1000, 0.01))`, which makes the internal search use these scaled coordinates. Returned parameter values remain in the original units. A log transformation, as in the Poisson example, can also help when positive parameters vary over several orders of magnitude.

Why the starting point matters. Nelder–Mead explores using the current simplex. On an objective with several valleys, different starts can lead to different local minima. Several starts provide a useful check: compare their final objective values, then examine the associated parameter estimates. Agreement increases confidence in the result but does not prove global optimality.

When to stop and what to inspect. A small triangle with similar values at its vertices suggests little further local improvement. Similar values can also occur on a flat region while the vertices remain far apart. R's Nelder–Mead routine uses an objective-value stopping criterion, so inspect the returned parameters as well as the completion code.

Without a known answer, check whether nearby trial points or additional starts find lower values. Nelder–Mead is mainly a local search method for a small number of parameters; it does not generally guarantee convergence to a minimizer.

7 Simulated annealing: sometimes move uphill

Imagine an objective with several valleys. The bottom of one valley is a *local minimum*: nearby points have larger objective values. A *global minimum* has the smallest objective value over the entire search region. A local minimum can therefore be a good answer within its neighborhood while another valley contains a much better answer.

With small steps, moving from one valley to another may require a temporary increase in the objective. A search that accepts only improvements can stop before crossing that barrier. **Simulated annealing allows some uphill moves**, giving the search a chance to leave its current valley. An uphill move is accepted to help exploration, rather than because it is immediately a better fit.

Separate the proposed move from the decision to accept it. Starting at the current point $\mathbf{x}$, propose a new point $\mathbf{y}$ and evaluate its objective. Define

$$\Delta = f(\mathbf{y}) - f(\mathbf{x}).$$

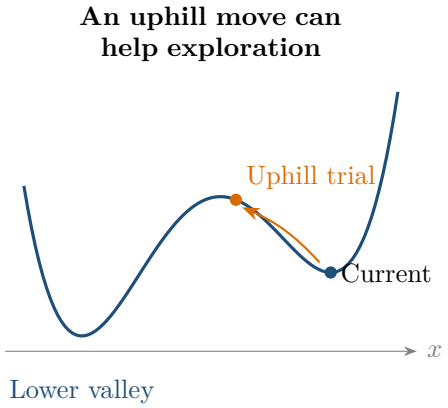
The sign of Δ tells us whether the proposal is an improvement: negative means downhill, positive means uphill, and zero means equal objective values. The size of a jump in parameter space and the resulting change in objective value are different quantities.

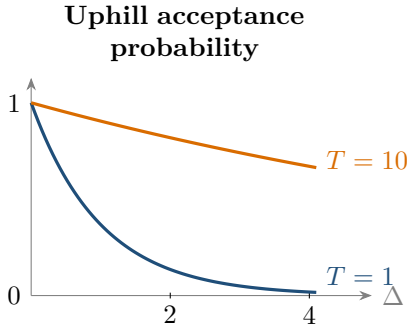
For a *symmetric proposal*, a jump from $\mathbf{x}$ to $\mathbf{y}$ is as likely to be proposed as the reverse jump from $\mathbf{y}$ to $\mathbf{x}$. The acceptance rule is then

$$\Pr(\text{accept } \mathbf{y} \mid \mathbf{x}) = \begin{cases} 1, & \Delta \le 0, \\ \exp(-\Delta/T), & \Delta > 0. \end{cases}$$

Here $T > 0$ is the *temperature*. Improvements are always accepted. For an uphill proposal, draw a uniform random number U between zero and one and accept if $U < \exp(-\Delta/T)$. On rejection, stay at $\mathbf{x}$ and propose again; the rejected point does not become the starting point for the next move.

Read the ratio Δ/T. A large uphill increase relative to T is unlikely to be accepted. A small increase relative to T is much easier to accept. Thus, temperature controls how much temporary deterioration the search is willing to tolerate. As T becomes small, acceptance of any fixed positive increase becomes rare.





Read the two pictures together. The left panel shows why accepting an uphill proposal can be useful: the current point is in the higher valley, and a move toward the ridge can help the search

explore the lower valley on the left. The right panel shows the acceptance probability for uphill changes $\Delta \geq 0$. Both curves begin at one, but the high-temperature curve decreases much more slowly. The algorithm uses the objective values to make these decisions; it does not know in advance where the lower valley is.

A numerical acceptance decision. Suppose the current objective is 5 and the proposed objective is 7, so $\Delta = 2$.

Temperature	Acceptance probability	Approximate value
$T = 10$	$\exp(-2/10)$	0.819
$T = 1$	$\exp(-2)$	0.135
$T = 0.2$	$\exp(-10)$	0.0000454

For example, if the uniform draw is $U = 0.4$, accept this proposal at $T = 10$ but reject it at $T = 1$. At either temperature, a decrease from 5 to 3 would be accepted automatically.

Temperature is meaningful relative to the scale of the objective. If we multiply the objective by 100, the changes Δ also become 100 times larger. Multiplying T by 100 preserves the acceptance probabilities for the same proposed moves.

The search loop.

1. **Initialize.** Choose a starting point and record its objective as both the current and best value.
2. **Propose.** Generate a new candidate from the current point and evaluate its objective.
3. **Accept or reject.** Accept improvements and use $\exp(-\Delta/T)$ for uphill proposals. If the proposal is rejected, keep the current point.
4. **Update the record.** If the new current value improves on the best recorded value, save both the point and its value.
5. **Cool and continue.** After a block of proposals, reduce the temperature according to a cooling schedule. Continue until the computational budget is used, then return the best record.

Current point and best point play different roles. The current point determines where the next proposal is generated. The best point preserves the most successful result. For example, one possible sequence is:

Event	Current objective	Best objective
Start	5	5
Accept a downhill move to 4	4	4
Accept an uphill move to 6	6	4
Reject an uphill trial at 7	6	4
Accept a downhill move to 3	3	3

The current value can rise and fall. The best recorded value can only stay the same or improve. After the accepted move to 6, the search continues from that point while retaining the earlier point with objective 4 as its best result.

Why cool gradually? Early in the search, a higher temperature encourages movement between regions. Later, a lower temperature favors improvements within the regions reached. Cooling too

quickly can restrict exploration before a better valley has been found. Remaining hot for too long uses the budget on movement with little improvement in the best record. The name comes from annealing a material: heating allows rearrangement, and gradual cooling encourages a low-energy configuration.

A useful computational strategy is to let annealing explore first, then apply a local method to the best point found. Annealing helps choose a valley; local refinement helps locate its bottom accurately.

8 Using SANN in R

R's `optim(..., method="SANN")` provides simulated annealing. We will use a deliberately simple proposal so that the two sources of exploration are easy to distinguish: the proposal determines *where to try*, and temperature determines *whether to accept* an uphill trial.

Setting or argument	What it changes
`temp`	Starting temperature; controls early acceptance of uphill changes relative to the scale of the objective.
`tmax`	Number of evaluations at each temperature before the next cooling update.
`maxit`	Total objective-evaluation budget for SANN.
`gr`	A function proposing a candidate point. In SANN this argument supplies proposals rather than derivatives.

A landscape with many valleys

Consider the Rastrigin function in two dimensions:

$$f(x_1, x_2) = \sum_{j=1}^{2} \left[x_j^2 + 10\{1 - \cos(2\pi x_j)\}\right].$$

The cosine terms create repeated bumps and valleys, while the quadratic terms make distant regions generally higher. Each summand is nonnegative, and both are zero at $(0, 0)$, so the global minimum is $f(0, 0) = 0$.

Starting at $(3, 3)$ gives an objective value of 18. A local method can settle near this starting valley even though the origin has a much smaller value. This makes the example useful for illustrating the difference between refining a valley and exploring several valleys.

Define the objective and proposal.

```
rastrigin <- function(x) {
  10 * length(x) + sum(x^2 - 10 * cos(2 * pi * x))
}

proposal <- function(x) {
  x + rnorm(length(x), sd = 0.7)
}

rastrigin(c(3, 3))             # 18
rastrigin(c(0, 0))             # 0
```

The proposal adds an independent normal perturbation to each coordinate. Its standard deviation 0.7 controls typical step size. Because the perturbations have mean zero and a symmetric distribution, the forward and reverse proposals are equally likely. The acceptance rule from the previous section therefore applies.

This custom proposal has a **fixed step scale**. Cooling changes uphill acceptance while the normal standard deviation remains 0.7. R's default proposal also depends on temperature; supplying this function keeps that extra dependence out of the example.

Explore, then refine. The function below performs one complete run for a chosen random seed. All runs start from the same point and use the same settings.

```
run_sann <- function(seed) {
  set.seed(seed)

  sa <- optim(c(3, 3), rastrigin, gr = proposal,
              method = "SANN",
              control = list(maxit = 20000, temp = 5, tmax = 20))

  nm <- optim(sa$par, rastrigin, method = "Nelder-Mead",
              control = list(maxit = 3000, reltol = 1e-12))

  best <- if (nm$value < sa$value) nm else sa

  c(annealing = sa$value,
    refined = best$value,
    x1 = best$par[1],
    x2 = best$par[2],
    evaluations = unname(sa$counts["function"] +
                         nm$counts["function"]))
}
```

The first `optim()` call explores the landscape. For SANN, `sa$par` and `sa$value` contain the best recorded point and its objective. The second call starts Nelder–Mead at that point to improve local accuracy.

The line defining `best` keeps the better result, including both its parameter vector and objective value. Thus, `refined` cannot exceed `annealing`, and the reported coordinates always correspond to the reported refined value. The evaluation count adds the cost of the two stages.

Repeat the randomized search. Setting a seed makes a run reproducible in the same R setup. Changing the seed changes the proposals and acceptance draws, so repeat the search to see how sensitive the result is to its random choices.

```
seeds <- c(11, 29, 47, 71, 101)
results <- t(vapply(seeds, run_sann, numeric(5)))
data.frame(seed = seeds, results)

local_only <- optim(c(3, 3), rastrigin, method = "Nelder-Mead",
                    control = list(maxit = 3000, reltol = 1e-12))

c(x1 = local_only$par[1],
  x2 = local_only$par[2],
  objective = local_only$value)
local_only$counts
```

Each row reports the best annealing value, the value retained after refinement, its two coordinates, and total evaluations. A value close to zero together with coordinates close to $(0, 0)$ indicates that a run reached the global valley accurately.

Compare the rows across *all* seeds. Local refinement can reduce the remaining error within the valley annealing found, but it can also refine a nonglobal valley. The local-only fit shows what happens when we refine directly from $(3, 3)$.

This comparison illustrates the roles of exploration and refinement. For a comparison at the same computational cost, give repeated local starts a comparable total objective-evaluation budget.

See where the searches finish

The following contour plot marks the starting point, the local-only result, and the retained results from all five annealing runs. It displays their locations rather than their full search paths.

```
grid <- seq(-4, 4, length.out = 240)
z <- outer(grid, grid, function(a, b) {
  20 + a^2 + b^2 - 10 * cos(2 * pi * a) - 10 * cos(2 * pi * b)
})

contour(grid, grid, z, levels = c(0.1, 1, 5, 10, 20, 30, 40),
        drawlabels = FALSE, asp = 1,
        xlab = expression(x[1]), ylab = expression(x[2]))
points(3, 3, pch = 19, col = "firebrick")
points(local_only$par[1], local_only$par[2],
       pch = 17, col = "darkorange")
points(results[, "x1"], results[, "x2"],
       pch = 4, col = "steelblue", cex = 1.2)
points(0, 0, pch = 8)
legend("topleft",
       c("Start", "Local only", "Annealing + local", "Global minimum"),
       pch = c(19, 17, 4, 8),
       col = c("firebrick", "darkorange", "steelblue", "black"),
       bty = "n", cex = 0.8)
```

Several retained points may overlap if different seeds find the same valley. The printed table helps distinguish a point near the origin from a solution with appreciable remaining objective error.

Understand the controls

Temperature and step size answer different questions. Increasing `temp` makes a given uphill change easier to accept early in the run. Increasing the proposal standard deviation makes more distant points likely to be proposed. A very small step can explore slowly; a very large step can repeatedly land in high regions. The value 0.7 is an illustrative choice for this landscape.

Cooling occurs in blocks. R uses a logarithmic cooling schedule. With `tmax=20`, a temperature is used for a block of evaluations before the next update. Increasing `tmax` changes the duration of those blocks; it does not increase the total budget when `maxit` stays fixed. With the custom proposal above, cooling leaves the proposal's standard deviation unchanged.

Budget and accuracy are different. Increasing `maxit` gives SANN more objective evaluations. It provides more opportunity to improve the best record, but does not specify how close that record

must be to a minimum. The Nelder–Mead refinement has its own separate budget and stopping settings.

Computing note. For SANN, a zero convergence code reports completion of the requested budget. The usual `reltol` accuracy stopping rule does not apply to SANN. In the example, `reltol=1e-12` is supplied only to the subsequent Nelder–Mead call.

9 Choosing a method and interpreting the answer

Start with the structure of the objective. How many parameters are there? Is there one clear valley or several? Is a single evaluation quick, or does it require a costly model fit or simulation? These features help determine which search is worth trying.

Method	Main idea	Useful setting
Trisection	Compare two interior points and discard a third.	One parameter with one clear valley.
Golden section	Reduce the interval while reusing a function value.	One parameter, especially when evaluations are costly.
Brent	Combine a parabolic guess with reliable interval reduction.	A practical scalar search; use `optimize()`.
Nelder–Mead	Move and reshape a simplex using objective values.	A few parameters and a local search.
SANN	Accept some uphill moves, then become more selective.	Several valleys and a larger evaluation budget.

For a scalar objective with one clear valley, interval reduction makes direct use of that structure. With a few parameters, a moving simplex offers a simple local search. With several competing valleys, annealing or repeated starts can help explore different regions. Annealing followed by local refinement combines these two roles.

Check four things before trusting a result.

1. **Location and value: what was actually found?**

   Inspect the parameter estimate and recalculate the objective there. Check that the coordinates satisfy the scientific requirements of the model. Keep the parameter vector and its value together when comparing candidate answers. A lower value identifies a better candidate only when the same objective is being compared.

2. **Search region: what locations were explored?**

   For an interval method, check that the interval covers the parameter values of interest. If the result lies near an endpoint, examine whether the optimum might be outside the chosen interval. For a local search, compare additional starting points. For annealing, compare several random seeds and examine how often similarly good values are obtained.

3. **Cost: how much work produced the answer?**

   Compare objective evaluations and elapsed time. One iteration may evaluate several trial points, and methods use the word *iteration* differently. For example, SANN's `maxit` specifies its evaluation

budget. A result from 20,000 evaluations has used more opportunity to explore than one from a few hundred. Timing also matters when evaluations vary in computational cost.

4. **Accuracy: does further effort materially change the answer?**

   For methods with an accuracy tolerance, tighten it and compare the returned point and objective. For SANN, increase the budget or add repeated runs. A stable objective with changing parameters can indicate a flat region: many parameter values may fit almost equally well. The required numerical accuracy should reflect the purpose of the analysis.

Handle constraints through the model or a suitable method. For a positive parameter, searching over its logarithm guarantees positive trial values, as in the Poisson example. For a parameter strictly between zero and one, a logistic transformation can play the same role. These transformations describe the feasible interior; a possible optimum on the boundary needs separate attention.

The `lower` and `upper` arguments do not add bound handling to Nelder–Mead or SANN. In `optim()`, supplying finite bounds with either method causes a warning and a switch to `L-BFGS-B`. Choose the method and parameterization deliberately.

Recognize noise in the objective. Some objectives are estimated using simulation. Evaluating the same parameter twice may then give different numbers. Before interpreting a tiny improvement as progress, compare it with the size of these fluctuations. Repeated evaluations or increased simulation effort can help assess a promising candidate. An extremely tight optimization tolerance is of little use when the objective itself is much noisier.

The main lesson. Under a one-valley assumption, interval methods can progressively rule out locations. Nelder–Mead reshapes a local search region. Annealing allows temporary deterioration to explore other regions, while keeping the best point found. Match the method to the problem, then inspect the returned location, its objective value, and its sensitivity to further search.

Further reading

The references below give implementation details and theory when needed. R's help pages describe method-specific controls: `help("optimize")` and `help("optim")`.

References

Brent, R. P. (1973). *Algorithms for Minimization without Derivatives*. Prentice-Hall.

Nelder, J. A. and Mead, R. (1965). A simplex method for function minimization. *The Computer Journal*, **7**, 308–313.

R Core Team (2026). R documentation for package `stats`: `optim` and `optimize`.